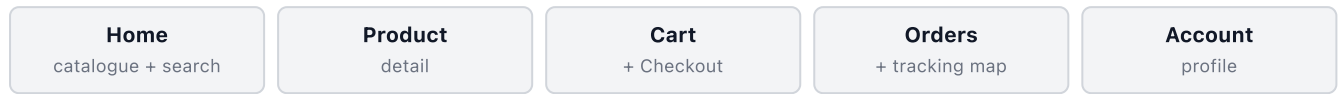


Gigabox Mini — Architecture

React Native 0.87 · TypeScript · Redux Toolkit · FlashList · Google Maps (WebView)

Layers

Screens — render state, dispatch actions. No business logic.



▼ dispatch / subscribe ▲

Redux Store — the single source of truth



Listener Middleware — side effects, running independently of any screen



External — outside the app



Why middleware instead of screen code?

An order must keep progressing when you're on another tab, or the app is closed. Putting that logic in a screen ties it to that screen being open. In the store, it survives navigation — and the map becomes a *display* of state, not the driver of it.

Two rules worth mentioning

1. Compute once. The discount, the delivery fee and the order total each live in exactly one function (`utils/price.ts`).

Three real bugs came from one number being calculated in two places and drifting — the cart quoted ₹50 while checkout charged ₹90.

2. Derive, don't store. An order saves one timestamp (`placedAt`). Its status is calculated from elapsed time.

So closing the app is safe — nothing has to keep running, because the answer comes from the clock.

Order Flow

What happens from tapping “Add” to seeing “Delivered”

1 · Browse → Cart



Home, product detail and cart all show the **same number** for a product, because all three read it from the cart.

2 · Checkout → Order created



3 · Delivery timeline



QUESTION	ANSWER
Who moves the status?	<code>orderProgress</code> middleware in the store — not the screen
If I switch tabs?	Keeps progressing. Nothing is tied to the screen being open.
If I close the app?	Timers die — but on reopen the status is recalculated from <code>placedAt</code> . Closed for 10 minutes → shows Delivered instantly.
What does the map do?	Only draws. It receives a <code>phase</code> (waiting / moving / delivered) and animates. It never decides when delivery completes.

4 · The tracking map



Worth mentioning if asked about tricky parts

Status changes are pushed into the running page with `injectJavaScript` rather than rebuilding its HTML. Rebuilding reloaded the whole map — which destroyed the animation and re-billed a Directions request on every status change.

Known limits (be upfront)

No backend — orders live on the device. Store and delivery locations are hardcoded, so every order draws the same route. Notifications arrive on reopen rather than while closed, because Android batches deferred background work; surviving a force-stop needs server-sent push.